

Guía del ETL para el equipo

Onboarding, ingesta de la Federación, receta para nuevas fuentes, glosario y reglas para agentes.

Generado el 29-09-2026 desde los Markdown del repositorio (fuente única): docs/etl/onboarding.md, docs/etl/federacion.md, docs/etl/nueva-fuente.md, docs/etl/glosario.md, etl/AGENTS.md, etl/README.md. Si cambias una regla, edita el Markdown y corre `make etl-docs`.

ETL de ChessQuery — onboarding

Guía para quien se suma a trabajar en `etl/`. En 10 minutos deberías tener el entorno andando y entender el recorrido de un dato de punta a punta. Términos en `docs/etl/glosario.md`.

1. Qué hace el ETL (en una frase)

Trae ratings y torneos de **fuentes externas** (FIDE y la Federación Chilena de Ajedrez), los **valida y minimiza**, los guarda en **S3** y avisa por **SNS** para que el servicio `users` (y más adelante `tournament`) los aplique. El ETL **nunca escribe en la base de datos**: publica eventos.

```
fuelle externa -> validar -> minimizar -> S3 staged/ -> diff vs corrida anterior -> SNS (lotes de 200) -> SQS -> users
                                     ↳ S3 rejected/ (solo id + motivo)                                     ↳ S3 manifests/
(conteos)
```

2. Entorno local en 10 minutos

Requisitos: Docker, Python 3.12+, Make.

```
make local-up           # Postgres, LocalStack (SNS/SQS/S3), mock OIDC, Mailpit
make etl-setup          # crea etl/.venv e instala el paquete + pytest
make test-etl           # tests del ETL (cobertura mínima 90 %)
make etl-fide-local     # lista FIDE real -> LocalStack (necesita `make users` corriendo
para verla en la BD)
make federation-contract # verifica el esquema de la Federación (no baja datos de
personas)
make federation-tournaments-local # torneos de la Federación en vivo -> LocalStack
```

Para ver el efecto en la base: `make users` en otra terminal y luego `http://localhost:8081/api/public/ranking?type=FIDE_STANDARD`.

3. Mapa de carpetas

Ruta	Qué es
<code>etl/chessquery_etl/pipeline.py</code>	Esqueleto común: <code>stage</code> , <code>changed_rows</code> (diff), <code>publish_batches</code> , <code>write_manifest</code> , <code>envelope</code>
<code>etl/chessquery_etl/fide.py</code>	Parser de la lista oficial de FIDE (columnas desde el encabezado)
<code>etl/chessquery_etl/handler.py</code>	Lambda y CLI de FIDE; adaptadores <code>S3Storage</code> y <code>SnsBus</code>
<code>etl/chessquery_etl/federation/</code>	Ingesta de la Federación (ver <code>docs/etl/federacion.md</code>)
<code>etl/tests/</code>	Tests con datos sintéticos ; <code>federation_fakes.py</code> tiene los dobles de prueba
<code>docs/events.md</code>	Contrato de los eventos que publicamos (<code>rating.updated</code> , <code>federation.tournament.published</code>)

4. Reglas que no se rompen (coordinar antes de cambiarlas)

1. **Contrato de eventos:** cualquier cambio en la forma de un evento se documenta primero en `docs/events.md` y se coordina con quien mantiene `users / tournament`.
2. **Lista blanca de campos** (`federation/contract.py`): define qué datos personales pedimos. Agregar un campo es una decisión de privacidad.
3. **El RUT nunca sale del proceso en claro:** se valida y se convierte en `rutHash` (`federation/privacy.py`). Si cambia ese cálculo, el match en `users` se rompe sin avisar: hay un test que lo compara con Java.
4. **El pepper no se cambia** sin recalcular los hashes existentes (vive en SSM; en local, `PRIVACY_PEPPER`).
5. **Descarga masiva de personas apagada** (`FEDERATION_BULK_PLAYERS_ENABLED=false`) hasta que exista convenio.
6. **Datos de prueba siempre ficticios:** nunca se commitean datos reales de personas (ni en fixtures ni en logs).

5. Cómo probar sin datos reales

Los tests usan transportes falsos (`FakeTransport`) y almacenamiento en memoria (`MemoryStorage` , `MemoryBus`): no hay red ni AWS. Para una fuente nueva, crea fixtures sintéticas con la **forma real** de la fuente (ver `tests/fixtures/players_list_sample.txt` , que tiene el encabezado real de FIDE con jugadores inventados).

6. Cómo se despliega

Cada fuente es una Lambda (`handler.lambda_handler` para FIDE, `federation.cli.lambda_handler` para la Federación), definida en el módulo Terraform `infra/terraform/modules/etl-jobs` :

Lambda	Cuándo corre	Qué hace
<code>fide-import</code>	día 2 de cada mes (regla de EventBridge)	lista FIDE (CHI) → <code>rating.updated</code>
<code>federation-tournaments</code>	todos los días	torneos de la Federación → <code>federation.tournament.published</code>
<code>federation-lookup</code>	cuando llega un mensaje a la cola <code>etl-federation-lookup</code>	ficha de un jugador que la vinculó → <code>rating.updated</code>

El código se empaqueta tal cual desde `etl/chessquery_etl` (solo usa `boto3` , que ya viene en el runtime de Lambda). El pepper para el hash del RUT se lee de SSM (`PRIVACY_PEPPER_PARAM`), nunca de una variable en texto plano. Se usan reglas de EventBridge y no Scheduler porque el Learner Lab no permite crear el rol que Scheduler necesita. En local todo se corre con `make` (ver `etl/README.md`).

7. Sigüientes lecturas

- `docs/etl/federacion.md` — la fuente más delicada: parámetros, validaciones y runbook.
- `docs/etl/nueva-fuente.md` — receta para agregar una fuente.
- `docs/etl/glosario.md` — vocabulario del equipo.

Ingesta de la Federación Chilena de Ajedrez

Código: `etl/chessquery_etl/federation/`. Fuente: el endpoint GraphQL público del sitio (<https://www.federacionchilenadeajedrez.cl/graphql>), el mismo que usa su página de jugadores. **No es una API oficial**: no tiene documentación ni términos de uso publicados, y su esquema puede cambiar sin aviso.

Qué se trae y cómo

Qué	Query GraphQL	Modo	Estado
Torneos publicados (datos del evento)	<code>rankedTournaments / tournaments(word)</code>	en vivo	✅ encendido
Un jugador por su id federativo	<code>person(id)</code>	consulta puntual, cuando la persona vincula su ficha	✅ encendido
Todos los jugadores por región y género	<code>personRanking(id, gender, elo, region)</code>	masivo	🚫 apagado hasta convenio
Historial de ELO por período	<code>periodPlayer / elosHistoric</code>	—	🕒 semántica por confirmar con la Federación

Por qué el masivo está apagado: la ficha de cada persona expone RUT, fecha de nacimiento, email y estado de pago de cuotas, incluidos menores. Sin convenio no hay una base legal clara para descargar miles de fichas (Ley 19.628 / 21.719). La consulta puntual sí tiene base: el propio titular la pide al vincular su id. Además, los argumentos de `personRanking` (`id`, `elo`) no están documentados: `FEDERATION_RANKING_ID` y `FEDERATION_RANKING_ELO` se completan cuando la Federación confirme su significado.

Consulta puntual: el jugador vincula su ficha

Es la única vía por la que hoy entran datos personales de la Federación, y siempre a pedido del titular:

1. En "Mi inicio", el jugador escribe su id federativo → `users` lo guarda en su cuenta y publica `federation.lookup.requested { playerId, federationId }`.
2. La cola `etl-federation-lookup` lo recibe. En la nube la atiende la Lambda `federation-lookup` (disparada por SQS); en local, `make federation-worker` (`federation/worker.py`, long polling).
3. Se corre el modo `lookup` para **solo ese id** (`person(id)`) → validación → minimización (RUT → `rutHash`, fecha → año) → `rating.updated` (source `FEDERACION`).
4. `users` encuentra la cuenta por el id federativo y completa ELO nacional, club y año de nacimiento.

Si la consulta falla, el mensaje vuelve a la cola (`batchItemFailures`) y tras 5 intentos pasa a la DLQ `etl-federation-lookup-dlq` (alarma). Un mensaje mal formado se descarta con un log: reintentarlo no lo arregla. Si la ficha ya existía en ChessQuery sin dueño, el jugador la **reclama** ("¿Eres tú?") verificando su RUT; eso lo resuelve `users` (evento `player.merged`), no el ETL.

Parámetros (variables de entorno)

Variable	Por defecto	Efecto
FEDERATION_BASE_URL	https://www.federacionchilenadeajedrez.cl	Sitio de origen (cambiar para pruebas contra un doble)
FEDERATION_RPS	1	Requests por segundo (máximo 5: es un sitio de terceros)
FEDERATION_TIMEOUT_S	30	Timeout por request
FEDERATION_MAX_RETRIES	3	Reintentos ante 429/5xx/red, con espera 1 s, 2 s, 4 s + jitter
FEDERATION_BREAKER_THRESHOLD	5	Fallos seguidos que abren el circuito y cortan la corrida
FEDERATION_USER_AGENT	ChessQuery-ETL/3 (+...; contacto@...)	Identificación con contacto (cortesía con el sitio)
FEDERATION_BULK_PLAYERS_ENABLED	false	Descarga masiva de personas. Solo con convenio firmado
FEDERATION_TOURNAMENTS_ENABLED	true	Ingesta de torneos
FEDERATION_REGIONS	all	Ids de región separados por coma, o all
FEDERATION_RANKING_ID / FEDERATION_RANKING_ELO	vacío	Argumentos de personRanking (por confirmar)
FEDERATION_MAX_REJECTED_RATIO	0.05	Si se rechaza más de esta proporción, la corrida no publica
PRIVACY_PEPPER (local) / PRIVACY_PEPPER_PARAM (nube, SSM)	—	Secreto del hash del RUT; debe ser el mismo que usa users
ETL_BUCKET , CHESS_EVENTS_TOPIC_ARN	local: chessquery-etl , tópico de LocalStack	Destinos S3 y SNS

Campos: qué pedimos y qué jamás

Lista blanca (contract.py), por tipo: - **Persona:** id, firstName, secondName, lastName, lastNameSecond, gender, title, fidelidentificator, eloNat, eloInter, birthdayFormated, identificator (RUT), clubBasic (id, nombre, región). - **Torneo:** id, title, city, startDate, endDate, type, rounds, timeControl, category, eloN, eloInt, club.

Nunca se piden (aunque existan): email , username , canon* (pagos de cuota), requestedChangeClub , flags de administración (PERSON_FORBIDDEN en contract.py).

Minimización antes de publicar (`players.to_event_player`): de la fecha de nacimiento solo queda el **año**; el RUT se valida (dígito verificador) y se publica como `rutHash` (HMAC-SHA256 con el pepper). El RUT en claro no llega ni a S3 ni a SNS.

Validaciones (motivos de rechazo)

Motivo	Regla
<code>id_invalido</code>	el id no es numérico
<code>nombre_vacio</code>	falta nombre o apellido
<code>rut_invalido</code>	tiene RUT pero el dígito verificador no calza
<code>elo_fuera_de_rango</code>	ELO distinto de 0 y fuera de 800–3000, o no numérico
<code>titulo_desconocido</code>	título fuera de GM, IM, FM, CM, WGM, WIM, WFM, WCM
<code>duplicado</code>	el mismo id aparece dos veces en la corrida
<code>torneo_sin_titulo</code> , <code>fecha_invalida</code> , <code>fin_antes_de_inicio</code> , <code>rondas_fuera_de_rango</code>	reglas de torneos

Los rechazos se guardan en `rejected/...jsonl` **solo con id y motivo**. Además, antes de bajar datos se verifica el **contrato**: si falta un campo de la lista blanca en el esquema, la corrida falla sin descargar nada.

Salidas en S3

<code>staged/source=federation-players/mode={lookup bulk}/period=AAAA-MM/players.jsonl</code>	
<code>staged/source=federation-tournaments/latest.jsonl</code>	← base del diff
<code>staged/source=federation-tournaments/date=AAAA-MM-DD.jsonl</code>	
<code>rejected/...jsonl</code>	← id + motivo
<code>manifests/...json</code>	← read, accepted, rejected, changed,
<code>batches</code>	

Runbook (qué hacer cuando falla)

Síntoma	Causa probable	Qué hacer
<code>ContractError</code> : Campos que ya no existen...	la Federación cambió su esquema	<code>make_federation_contract</code> , comparar con <code>contract.py</code> , ajustar la query y la lista blanca (con revisión de privacidad)
<code>CircuitOpenError</code>	el sitio respondió mal varias veces seguidas	esperar y reintentar; revisar si el sitio está caído; no subir <code>FEDERATION_RPS</code>
<code>RejectionThresholdError</code> (> 5 %)	cambió el formato de un campo	revisar <code>rejected/</code> (motivos agregados), ajustar la regla o el parser; no subir el umbral sin entender la causa
<code>HTTP 4xx</code> sin reintento	consulta inválida o bloqueo	revisar la query; contactar a la Federación si es un bloqueo
<code>BulkDisabledError</code>	se pidió el masivo con el flag apagado	es lo esperado sin convenio

Receta: agregar una fuente nueva al ETL

Ejemplo: sincronizar ratings de otra plataforma o una federación regional. Seguir los pasos en orden; cada uno tiene su verificación.

1. Entender la fuente (antes de escribir código)

- ¿Es oficial, tiene términos de uso? ¿Qué **base legal** tenemos para tratar esos datos (fuente pública, consentimiento del titular, convenio)? Si hay datos de personas, revisarlo con el equipo antes de seguir.
- ¿Qué campos trae? Arma la **lista blanca**: solo lo necesario para el producto.
- ¿Qué identificador estable tiene cada registro? (id propio, FIDE id, RUT). **Nunca** el nombre.

2. Contrato del evento

Si la fuente produce ratings, se reutiliza `rating.updated` con un `source` nuevo. Si produce otra cosa, se define un evento nuevo. En ambos casos: **documentar primero en** `docs/events.md` y coordinar con el consumidor.

3. Parser o cliente

- Archivo propio en `etl/chessquery_etl/` (o un subpaquete si la fuente es grande, como `federation/`).
- Leer por nombre de columna/campo, no por posición fija.
- Un cliente HTTP a un sitio de terceros siempre con ritmo máximo, reintentos y circuit breaker (reutilizar el patrón de `federation/client.py`).

4. Validación y minimización

- Reglas chicas, una por función, que devuelvan el motivo del rechazo (ver `federation/validation.py`).
- Minimizar antes de publicar: año en vez de fecha, `rutHash` en vez de RUT (`federation/privacy.py`).
- Umbral de rechazos: si se supera, la corrida no publica.

5. Orquestación con el pipeline común

Reutilizar `pipeline.stage`, `pipeline.changed_rows`, `pipeline.publish_batches` y `pipeline.write_manifest` (ver `federation/ingest.py`). No reimplementar el diff ni el envelope.

6. Tests (cobertura ≥ 90 %)

- Fixtures **sintéticas** con la forma real de la fuente (nunca datos de personas reales).
- Probar: parser, cada regla de validación, minimización (que el dato sensible no salga), diff (segunda corrida sin cambios publica 0), umbral de rechazos y el cliente con transporte falso.

7. Lado consumidor (`services/users`)

Si es una fuente de ratings nueva: una implementación de `RatingSource` que declare su `source` en `sources()` (ver `FederatedRatingSource` y `LinkedAccountRatingSource`). El consumer no se toca.

8. Operación

- Flag para encender/apagar la fuente y variables documentadas en una tabla (como `docs/etl/federacion.md`).
- Entrada en el runbook con los errores esperables.
- Lambda + EventBridge Scheduler en el módulo Terraform `jobs` .

Checklist de privacidad

- ☐ Base legal identificada (fuente pública, consentimiento o convenio).
 - ☐ Lista blanca de campos; campos prohibidos listados explícitamente.
 - ☐ RUT → `rutHash` ; fecha de nacimiento → año; sin emails ni datos de pago.
 - ☐ `rejected/` y logs sin datos personales.
 - ☐ La supresión (`data_suppression`) se respeta del lado de `users` .
-

Glosario del ETL

Término	Significado en ChessQuery
Fuente	Origen externo de datos: FIDE, la Federación, Lichess, Chess.com.
Federado / no federado	Jugador inscrito en la Federación (tiene id federativo y ELO nacional) o no. Los no federados son la mayoría y no aparecen en ninguna lista externa.
Período	Mes al que corresponde una lista de ratings, en formato AAAA-MM .
Lista blanca	Campos que sí pedimos a una fuente. Lo que no está en la lista no se descarga.
Minimización	Guardar y publicar solo lo necesario: año en vez de fecha de nacimiento, hash en vez de RUT.
Hash con pepper (rutHash)	HMAC-SHA256 del RUT normalizado con un secreto (pepper) que vive fuera de la base. Permite comparar RUT sin guardarlos en claro.
Supresión	Derecho de una persona a que borremos sus datos. Sus identificadores quedan en data_suppression para que ninguna fuente los vuelva a importar.
Consulta puntual (consentida)	Traer la ficha de una sola persona porque ella misma lo pidió (al vincular su id federativo).
Flag de masivo	FEDERATION_BULK_PLAYERS_ENABLED : habilita la descarga de todas las personas. Apagado hasta el convenio.
Contrato	Forma acordada de un evento (en docs/events.md) o del esquema de la fuente (en contract.py). Romperlo rompe a otros.
Staged	Copia validada y minimizada de una corrida, guardada en S3 para compararla con la próxima.
Diff	Comparación contra la corrida anterior: solo se publica lo nuevo o modificado.
Lote	Grupo de hasta 200 registros por mensaje SNS (el límite de un mensaje es 256 KB).
Manifest	Resumen de una corrida: leídos, aceptados, rechazados, cambiados, lotes.
Rechazo	Registro que no pasó una regla de validación. Se guarda solo con id y motivo.
Umbral de rechazos	Proporción máxima de rechazos (5 %) antes de detener la corrida sin publicar.
Idempotencia	Procesar dos veces el mismo evento no cambia el resultado (processed_event en users).
DLQ	Cola de mensajes que fallaron 5 veces; si recibe algo, suena una alarma.
Circuit breaker	Corte automático tras varios fallos seguidos para no insistir contra un sitio caído.
Envelope	Sobre común de todo evento: eventId , eventType , timestamp , payload .

Instrucciones para agentes que trabajan en `etl/`

Contexto completo: `docs/etl/onboarding.md`, `docs/etl/federacion.md`, `docs/etl/nueva-fuente.md`, `docs/etl/glosario.md` y el contrato de eventos en `docs/events.md`. Reglas generales del repo: `CLAUDE.md` en la raíz.

Reglas (no romper sin acuerdo explícito del equipo)

1. **Contrato de eventos:** no cambiar la forma de `rating.updated` ni de `federation.tournament.published` sin actualizar antes `docs/events.md` y avisar a quien mantiene `services/users`.
2. **Lista blanca de campos** (`chessquery_etl/federation/contract.py`): no agregar campos de personas sin revisión de privacidad. Nunca pedir `email`, `canon*` (pagos) ni flags de administración.
3. **RUT:** nunca sale del proceso en claro. Validar DV y publicar solo `rutHash` (`federation/privacy.py`). No cambiar el cálculo del hash: debe coincidir con `IdentifierHasher.java` (test `test_hash_coincide_con_java`).
4. **Fecha de nacimiento:** publicar solo el año.
5. **Descarga masiva de personas:** `FEDERATION_BULK_PLAYERS_ENABLED` queda en `false` por defecto; no activarlo en código, CI ni infraestructura.
6. **Datos de prueba:** siempre sintéticos (nombres y RUT ficticios). Nunca commitear datos reales ni imprimirlos en logs.
7. **Sitios de terceros:** todo cliente HTTP con ritmo máximo, reintentos con backoff y circuit breaker.
8. **Rechazos y logs:** solo id + motivo, sin datos personales.

Cómo trabajar

- Reutilizar el esqueleto de `chessquery_etl/pipeline.py` (`stage`, `changed_rows`, `publish_batches`, `write_manifest`, `envelope`); no reimplementar diff ni envelope.
- Funciones chicas: complejidad ciclomática ≤ 10 (`make complexity`).
- Comentarios, docstrings y mensajes en español latino, explicando el *por qué*.
- Tests: `make test-etl` (pytest, cobertura $\geq 90\%$). Sin red en los tests: usar `tests/federation_fakes.py`.
- Commits cortos, en español y con foco funcional, sin trailers de co-autoría.

etl — ingesta de ratings y torneos

Trae datos de fuentes externas, los valida y minimiza, los deja en S3 y publica eventos en el tópico SNS `chess-events`. **No escribe en la base de datos:** `users` consume los ratings y `tournament` el calendario de la Federación.

Empieza por `docs/etl/onboarding.md`. Guía en PDF: `docs/etl/ChessQuery-ETL-guia-equipo.pdf`.

Fuente	Módulo	Qué publica	Estado
FIDE (lista oficial mensual, federación CHI)	<code>fide.py</code> + <code>handler.py</code>	<code>rating.updated</code> (source <code>FIDE</code>)	✅ probado con la lista real: 9.271 leídos, 4.192 con rating
Federación Chilena de Ajedrez — torneos	<code>federation/</code>	<code>federation.tournament.published</code>	✅ en vivo
Federación — jugador puntual (lo pide el jugador al vincular su ficha)	<code>federation/worker.py</code>	<code>rating.updated</code> (source <code>FEDERACION</code> , con <code>rutHash</code>)	✅
Federación — jugadores masivo	<code>federation/</code>	<code>rating.updated</code>	🚫 apagado hasta convenio

```
make etl-setup           # venv + dependencias
make test-etl            # pytest (cobertura ≥ 90 %)
make etl-fide-local      # FIDE real → LocalStack
make federation-contract  # verifica el esquema de la Federación
make federation-tournaments-local # torneos de la Federación → LocalStack
make federation-worker    # atiende las fichas que piden los jugadores (cola etl-
federation-lookup)
```

En la nube cada fuente es una Lambda (módulo Terraform `etl-jobs`); ver `docs/etl/onboarding.md` §6.

Documentación: `docs/etl/onboarding.md` · `docs/etl/federacion.md` · `docs/etl/nueva-fuente.md` · `docs/etl/glosario.md` · contrato de eventos en `docs/events.md`.